49. Multi-Layer Perceptrons

If a single neuron is powerful, the network is even more resourceful.

Previous Section:

48. Neural Networks

Contents:

1. Single-Layer Perceptron
2. From a Single Perceptron to Neural Networks
3. From Perceptron to Multi-Layer Perceptron
4. Multi-Layer Perceptron — A Concrete Example
5. Backpropagation — How the Network Learns
5. Backpropagation — Computing the Weights
6. Backpropagation — Updating the Parameters
7. Multi-Layer Perceptron using Scikit-Learn
Summary
References

In the previous section, we explored neural networks in general—the idea of layers, neurons, and how information flows through them. But every concept has a starting point. Just like in probability, where the Binomial distribution reduces to a Bernoulli case when there is only a single trial, neural networks also begin from a simpler form.

At the very beginning, there is the **Perceptron**—more precisely, the **Single-Layer Perceptron**. This is the most basic form of a neural network: a model with no hidden layers, only inputs directly connected to an output. It represents the earliest attempt to mimic how a neuron makes a decision.

But as we move forward, things become more interesting. When we stack multiple layers of these perceptrons together, we get what is called a **Multi-Layer Perceptron (MLP)**. This is where neural networks begin to show real power. Unlike the single-layer version, MLPs can learn complex, non-linear relationships—problems that cannot be solved with a simple straight line. A classic example is the XOR problem, where the model must learn a pattern that is not linearly separable.

Now here comes an important clarification. You might wonder: *Why do we even use the term "Multi-Layer Perceptron"? Why not just call everything a Neural Network?*

The answer is subtle but important. "Neural Network" is a broad, umbrella term. It includes many different architectures: Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Transformers, and more. Each of these is designed for a specific type of problem.

"Multi-Layer Perceptron," on the other hand, is a **specific type of neural network**. It refers to the standard, fully connected, feedforward architecture where every neuron in one layer connects to every neuron in the next.

So while every MLP is a neural network, not every neural network is an MLP. And just like understanding Bernoulli helps us understand Binomial, understanding the Single-Layer Perceptron helps us understand

everything that comes after.

1. Single-Layer Perceptron

(Sometimes, we just call it a perceptron)

Every complex idea has a simple beginning. And for neural networks... that beginning is the **Perceptron**. This is the most basic form of a neural network. No deep layers. No complicated structure. Just a single unit making a decision.

At its core, a perceptron does something very straightforward: It takes inputs, combines them, and decides: 0 **or** 1**.** That's it. But don't let that simplicity fool you, because this tiny idea is what everything else is built on.

How does a perceptron actually work?

Imagine you are trying to make a decision. You don't treat every piece of information equally, right? Some things matter more. Some matter less. That's exactly how a perceptron thinks.

The perceptron receives multiple inputs, assigns importance to each input, combines them into a single value, applies a rule, produces a decision. And because the output is always **0 or 1**, perceptrons are naturally used for **binary classification**.

Core Components of a Perceptron

Let's break it down. Piece by piece.

$$\text{Inputs} = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ \ldots, \boldsymbol{x}_n)$$

These are the features, the information we feed into the model. Think of them as signals. For example, in a simple logical OR problem: $(x_1, x_2) \in 0, 1$

By themselves, inputs don't decide anything. They are just raw information. That comes the weights:

$$\text{Weights} = (\boldsymbol{w}_1, \boldsymbol{w}_2, \ \ldots, \boldsymbol{w}_n)$$

Now this is where things become interesting. Weights tell us: **How important is each input?** Large weight → strong influence; Small weight → weak influence.

During training, these weights are adjusted. So over time, the model learns: "This input matters more than that one."

$\text{Bias } (\boldsymbol{b})$ **-** Bias is a small detail, but a very powerful one. It allows the model to shift its decision. Without bias, the perceptron is too rigid. With bias, it gains flexibility.

A good way to think about it: Weights control the **direction** of the decision boundary and bias controls the **position.**

Net Input (Weighted Sum) - Now we combine everything together:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

This value z represents the total "evidence" collected from all inputs. Before making a decision... the perceptron asks: "Is this value strong enough?"

Activation Function (Step Function) - Finally, the decision.

The perceptron applies a simple rule: If $z \geq 0$ → output **1;** Otherwise → output **0.**

This is called a **step function**. No probabilities. No uncertainty. Just a hard decision.

What does this really mean? A perceptron is essentially drawing a line (or a hyperplane in higher dimensions) to separate data into two groups.

- Everything on one side → class 1;
- Everything on the other → class 0.

It's simple. Clean. Logical. But also... limited. Because it can only solve problems that are **linearly separable**. And that limitation? That's exactly why we need something more powerful.

Because every neural network you'll ever see, every deep learning model, every advanced architecture. They all come from this one idea:

> **Inputs → Weights → Sum → Activation → Output**

The perceptron is not the end. It's the starting point.

2. From a Single Perceptron to Neural Networks

A single perceptron is powerful, but only to a point. It can draw a line. It can separate simple patterns. But the moment the problem becomes even slightly complex, it breaks.

So what do we do? We don't replace it. We **stack it**. And that simple idea, connecting many perceptrons together, is what gives us a **Neural Network**.

Instead of one neuron making one decision, we build layers of neurons. Each layer learns something... and passes it forward. Step by step, the network transforms raw data into understanding.

The Structure of a Neural Network - A standard neural network has three main parts:

1. **Input Layer — "Just pass the information"**

This is where everything begins. We feed the model a feature vector:

$$x = (x_1, x_2, ..., x_n)$$

And that's it. No learning. No transformation. Just passing the data forward.

2. **Hidden Layers — "Where learning actually happens"**

This is the heart of the network. Each hidden layer takes what came before, and transforms it into something more meaningful. The computation looks like this:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \sigma(z^{(1)})$$

Here's what's happening behind the scenes:

- $W^{(1)}$: weights → what to pay attention to
- $b^{(1)}$: bias → how to shift decisions
- σ : activation → how to transform the signal

This activation is critical. Without it, the entire network collapses into something linear. With it, the model can learn **non-linear relationships**. That's the difference between drawing a straight line and understanding real-world patterns

Common activation functions include: ReLU, Sigmoid, Softmax, Tanh. Each one changes how the network "thinks".

3. **Output Layer — "Make the final decision"**

After passing through all hidden layers... we arrive here. The final computation:

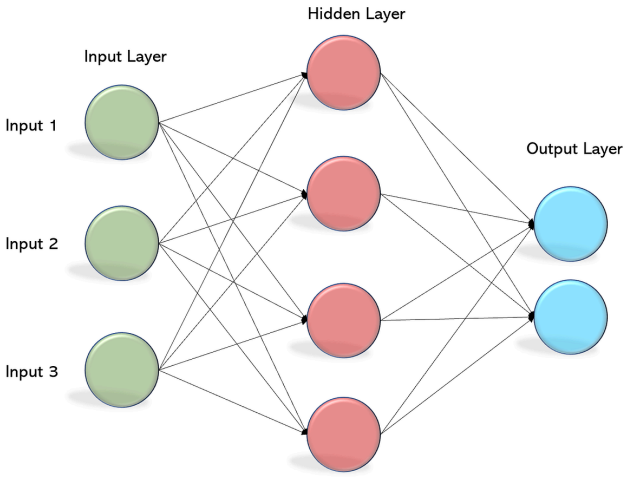
$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad \hat{y} = \sigma(z^{(2)})$$

This is where prediction happens. And depending on the problem, the activation changes:

- **Sigmoid** → Binary classification
- **Softmax** → Multi-class classification
- **Linear** → Regression

So the same structure adapting to completely different tasks.

(The image is used for reference only)



Why does stacking layers matter? Because now, the network is no longer limited to a straight line. It can bend, curve, adapt. It can learn **complex decision boundaries**. That's something a single perceptron can never do.

Simplified Learning Workflow - Now let's zoom into how a perceptron (and by extension, a neural network) learns. At each step, it follows a simple cycle:

1. Combine inputs: $z = \sum_{i=1}^{n} w_i x_i + b$

2. Apply activation: Turn the raw value into a prediction.

3. Measure error: How far off were we? $\text{error} = y - \hat{y}$

4. Update weights: $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$

5. Update bias: $b \leftarrow b + \eta(y - \hat{y})$. Here, η is the learning rate → how fast we adjust.

6. Repeat: We do this again, and again...,and again. Across all samples. Across multiple iterations (epochs). And slowly, the model improves.

7. Final decision: $\hat{y} = \text{step}(w^T x + b)$

What's really happening here? The model is not memorizing. It is adjusting itself, little by little. Trying to reduce mistakes. Trying to get closer to the truth.

And this is the key transition: A **single perceptron** makes one simple decision. A **neural network** makes decisions in layers. In that layered structure, simple computations become powerful intelligence.

3. From Perceptron to Multi-Layer Perceptron

A single perceptron can draw a line. That's useful, but also very limiting. Because real-world problems are rarely that simple. Patterns are messy. Relationships are not straight. And data doesn't neatly separate into two halves with a clean boundary.

So what do we do? We don't change the perceptron itself. We build on top of it.

Instead of using one perceptron, we stack many of them. Layer by layer. And this is where we get something much more powerful: The Multi-Layer Perceptron (MLP)

What is an MLP, really? At its core, an MLP is just a neural network made of fully connected layers. Each layer takes the output from the previous one, transforms it, and passes it forward. Step by step, the data evolves. From raw numbers, to meaningful representation, to final prediction.

Why does this work? Because of two things:

- Layering → allows step-by-step transformation
- Non-linearity → allows the model to learn complex patterns

Without hidden layers, the model stays linear. With hidden layers, the model becomes expressive. That's the entire leap.

The Structure of a Multi-Layer Perceptron

Even though MLPs can become very deep, their structure is always built from the same three parts:

1. Input Layer — "Start with raw data"

This is where the model receives information. Each neuron represents one feature.

- 3 features → 3 neurons
- 10 features → 10 neurons

No computation happens here. It simply passes the data forward.

2. Hidden Layers — "Transform and understand"

This is where everything changes. Each hidden layer receives inputs, applies weights and bias, passes through an activation function And produces a new representation. Not just a transformation, but a refinement.

Each layer learns something different: Early layers learn simple patterns; Deeper layers learn more abstract relationships.

This is how the network "understands" data.

3. Output Layer — "Make the decision"

At the end, the network must decide. The structure depends on the problem:

- 1 neuron → binary classification
- Multiple neurons → multi-class classification
- Continuous output → regression

Same model. Different purpose. **Fully Connected**

In an MLP, every neuron connects to every neuron in the next layer. No shortcuts. No missing links. This is called a fully connected (dense) network. Each connection has a weight. And that weight answers a simple question: "How important is this signal?"

As data flows through the network: Each layer reshapes it. Each neuron adjusts it. Each activation transforms it. Until finally, the model produces an answer. Not by memorizing. But by learning relationships.

A perceptron gives us a single decision. An MLP gives us a process of decisions. Layer after layer, transformation after transformation, until simple building blocks become something powerful. And that's the moment where neural networks stop being just equations, and start becoming intelligence.

4. Multi-Layer Perceptron — A Concrete Example

So far, everything we've discussed has been abstract. Layers. Neurons. Transformations. Now let's make it real.

We will use a small dataset of 10 samples to understand how a **Multi-Layer Perceptron (MLP)** actually works. Each sample describes system behavior using five features, and the goal is simple: **Is this malware (1) or benign (0)?**

Sample	prio	static_prio	normal_prio	policy	vm_pgoff	class
1	120	120	120	0	1024	0
2	110	110	110	1	4096	1
3	115	115	115	0	2048	0
4	100	100	100	1	8192	1
5	125	120	125	0	512	0
6	105	105	105	1	6000	1
7	118	118	118	0	1500	0
8	102	102	102	1	7000	1
9	122	120	122	0	900	0
10	108	108	108	1	5000	1

We are not programming rules like: "If $\mathrm{vmpgoff} > 5000 \rightarrow$ malware". Instead... the model will **discover patterns on its own**.

MLP Architecture for This Dataset

- **Input Layer — "5 features → 5 neurons"**

Each feature becomes one neuron: $(\mathrm{prio}, \mathrm{static\_prio}, \mathrm{normal\_prio}, \mathrm{policy}, \mathrm{vm\_pgoff})$

No computation happens here. Just forwarding values.

- **Hidden Layer — "Where patterns are discovered"**

We assume **4 hidden neurons** (this is a design choice, not fixed). Each neuron may learn something different:

Hidden Neuron	Possible Pattern Learned
h_1	Relationships between scheduling priorities
h_2	Memory usage behavior
h_3	Interaction between policy and vm_pgoff
h_4	Unusual or abnormal priority patterns

These are not programmed. They are **learned automatically**.

- **Output Layer — "Final decision"**

We use **1 neuron** because this is binary classification. The output is a probability (using Sigmoid): 0 → benign; 1 → malware

Output Value	Interpretation
0.10	Very likely benign
0.52	Uncertain
0.91	Very likely malware

Step 1 — Computing the Weighted Sum

Everything starts the same way as a perceptron. Each neuron computes:

$$z = w^T x + b$$

Example: Sample 1

Input vector: $x = (120, 120, 120, 0, 1024)$

Assume weights for hidden neuron h_1 :

$$w = (0.02,\ 0.01,\ 0.015,\ 0.5,\ 0.0005), \quad b = 0.1$$

Now compute:

$$z_1 = (0.02)(120) + (0.01)(120) + (0.015)(120) + (0.5)(0) + (0.0005)(1024) + 0.1 = 6.012$$

This value represents the **activation strength before activation**.

Step 2 — All Hidden Neurons

Each neuron has its own weights. So we compute:

$$z_1 = w_1^T x + b_1; \quad z_2 = w_2^T x + b_2; \quad z_3 = w_3^T x + b_3; \quad z_4 = w_4^T x + b_4$$

Each neuron sees the same input, but interprets it differently.

Step 3 — Repeat for All Samples

We don't do this once. We do it for every sample:

$\text{Sample } 1 \rightarrow (z_1^{(1)}, z_2^{(1)}, z_3^{(1)}, z_4^{(1)})$

$\text{Sample } 2 \rightarrow (z_1^{(2)}, z_2^{(2)}, z_3^{(2)}, z_4^{(2)})$

$\ldots$

$\text{Sample } 10 \rightarrow (z_1^{(10)}, z_2^{(10)}, z_3^{(10)}, z_4^{(10)})$

General form:

$$z_k^{(j)} = w_k^T x^{(j)} + b_k$$

Where: k → neuron index; j → sample index

Step 4 — Step Matrix Representation (How it's actually done)

In practice, we don't compute one by one. We use matrices: $Z = XW + b$

Matrix	Description	Shape
X	Input data	10×5
W	Weights	5×4
Z	Hidden layer output	10×4

Resulting Matrix

$$Z = \begin{bmatrix} z_1^{(1)} & z_2^{(1)} & z_3^{(1)} & z_4^{(1)} \\ z_1^{(2)} & z_2^{(2)} & z_3^{(2)} & z_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ z_1^{(10)} & z_2^{(10)} & z_3^{(10)} & z_4^{(10)} \end{bmatrix}$$

What does this mean? Each **row** → one sample; Each **column** → one neuron

So instead of thinking: *"One neuron, one sample..."* → We think: ***"All neurons, all samples... at once."***

This entire process is just: ***Multiply*** → ***Add*** → ***Transform***

Again and again, layer by layer. But something incredible happens: Simple math becomes pattern recognition. Repeated transformations become intelligence. And that is the real power of a Multi-Layer Perceptron.

In practice, this matrix computation is implemented efficiently using Python, with NumPy being the ideal candidate.

- **Reading and Preparing the Data**

```python
# Perceptrons in Neural Network
import pandas as pd
import numpy as np

# Create dataset
data = {
    "prio": [120,110,115,100,125,105,118,102,122,108],
    "static_prio": [120,110,115,100,120,105,118,102,120,108],
    "normal_prio": [120,110,115,100,125,105,118,102,122,108],
    "policy": [0,1,0,1,0,1,0,1,0,1],
    "vm_pgoff": [1024,4096,2048,8192,512,6000,1500,7000,900,5000],
    "class": [0,1,0,1,0,1,0,1,0,1]
}

df = pd.DataFrame(data)

# Split features and target
X = df.drop(columns="class")
y = df["class"]

# Convert to NumPy arrays
X = X.values
y = y.values

# Normalize features
X = (X - X.mean(axis=0)) / X.std(axis=0)

print("Normalized Input Matrix X:\n", X)
```

This input matrix X is what the neural network "sees". Each row represents one sample, each column represents one feature.

We transformed raw data into a **numerical matrix**. But more importantly, we **normalized the features**. Why? Because not all features are equal. **vm_pgoff** → very large values; **policy** → very small values

Without normalization: Large-scale features dominate; small features become irrelevant; learning becomes unstable. Normalization fixes this. Now every feature contributes fairly.

- **Weighted Sum Computation**

```
# Weights: 4 neurons × 5 features
W = np.array([
    [0.02, 0.01, 0.015, 0.5, 0.0005],
    [0.03, 0.02, 0.01, 0.4, 0.0003],
    [0.01, 0.015, 0.02, 0.6, 0.0007],
    [0.025, 0.02, 0.012, 0.45, 0.0004]
])

# Bias for each neuron
b = np.array([0.1, 0.1, 0.1, 0.1])

# Initialize output matrix
Z = np.zeros((len(X), len(W)))

# Compute weighted sum
for i in range(len(X)):          # each sample
    for j in range(len(W)):      # each neuron
        z_k = np.dot(W[j], X[i]) + b[j]
        Z[i][j] = z_k

print("Output Matrix Z:\n", Z)
```

Each neuron produces: $z = w^T x + b$

The code will produce a matrix with Rows → samples (10) and Columns → neurons (4). Each value responses of one neuron to one sample This matrix Z is the **hidden layer before activation**.

At this stage, the network is still **linear**. If we stop here: The model is just a linear transformation. It cannot learn complex patterns. We need something more.

- **Activation Function**

```
# Sigmoid activation function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Apply sigmoid element-wise
H = sigmoid(Z)

print("Hidden Layer Output (After Sigmoid):\n", H)
```

We introduce **non-linearity**. Without this, neural networks are useless. Here, we use the **Sigmoid Function:** $\sigma(z) = \frac{1}{1+e^{-z}}$

H is the **hidden layer output**. It has the same shape as $Z \rightarrow (10 \times 4)$. But now transformed Each value now represents: How strongly a neuron "activates" for a given input.

Look at what just happened. We didn't write rules, define conditions, hardcode decisions. Instead, we built a system that transforms data, learns representations, and prepares for decision making.

This hidden layer is not the final answer. It's something more important: It creates **features that didn't exist before**

And in the next step, those features will be used by the **output layer** to make the final prediction.

5. Backpropagation — How the Network Learns

Up until now, the network can **produce outputs**. But producing outputs is not intelligence. Learning begins when the model asks:

> *"How wrong am I... and how do I fix it?"*

In a **Multi-Layer Perceptron (MLP)**: $\text{Input} \rightarrow \text{Hidden} \rightarrow \text{Output}$

We can compute the final prediction: $\hat{y}$. And compare it with the true label: y. So the simplest idea is: $\text{error} = y - \hat{y}$

But here's the problem, only the **output layer** sees this error. The **hidden layer is blind**. It doesn't know how much it contributed and whether it helped or hurt. So we need a way to send the error **backward and** distribute responsibility across all layers. That process is called **Backpropagation.**

Output Layer Computation

Before learning, we must complete the forward pass. The output neuron takes **hidden layer outputs** as input.

$$z_{\text{out}}^{(i)} = w_1 h_1^{(i)} + w_2 h_2^{(i)} + w_3 h_3^{(i)} + w_4 h_4^{(i)} + b_{\text{out}}$$

For example: For sample 1 (`z_out[0] = 0.8488589; y_hat[0] = 0.70032772`). The model predicts **70% probability of malware → But suppose: True label = 0 (benign), t**hen the model is likely **confident, and relatively wrong.**

You might think: $\text{error} = y - \hat{y}$. But this is not enough. This simple error fails miserably. Suppose that:

True	Prediction	Error
0	0.55	-0.55
0	0.99	-0.99

Both are wrong. But the second one is **much worse**. Yet the error only increases linearly. This is why neural networks use a **loss function**.

Binary Cross-Entropy (BCE)

For binary classification:

$$L = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

- Small mistakes → small penalty
- Confident wrong predictions → **huge penalty**

For regression problems, a common metric is Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

For sample 1 ($\hat{y} = 0.70032772; y = 0$):

$$L = -\log(1 - 0.70032772) = -\log(0.29967228) \approx 1.20507$$

Partially large loss → strong correction needed

This process is the same for all samples in the dataset:

- For each sample i, we compute the output weighted sum:

$$z_{out}^{(i)} = W_{out} \cdot H^{(i)} + b$$

- Compute the predicted probability:

$$y^{(i)} = \sigma(z_{out}^{(i)})$$

- Compute the loss for that sample:

$$L^{(i)} = -[y_i log(\hat{y}_i) + (1 - y_i) log(1 - y_i)]$$

- Then we average loss across every sample:

$$L = \frac{1}{n} \sum_{i=1}^{n} L^{(i)}$$

This value represents how well the neural network is performing overall.

The Backpropagation algorithm begins

Now everything connects. We compute prediction → compute loss → compute gradients → send error backward → update weights.

Gradients Needed

- Output layer: $\frac{\partial L}{\partial W_{out}}$
- Hidden layer: $\frac{\partial L}{\partial W_{hidden}}$

The Goal of Learning - We define the objective: Minimize the loss. Thus, **Gradient Descent is introduced:**

$$w = w - \eta \frac{\partial L}{\partial w}$$

- w → weights; η → learning rate; $\frac{\partial L}{\partial w}$ → gradient

This is the key idea: Move weights in the direction that **reduces loss.**

The network made a prediction, measured how wrong it was, traced the error backward, adjusted itself. No rules. No instructions. Just: Feedback → Correction → Improvement

This is the moment where a neural network stops being a mathematical structure and becomes a learning system.

5. Backpropagation — Computing the Weights

Now, everything comes together. We are no longer just passing signals forward. We are **closing the loop**. From prediction → to error → to correction. This is where the network **actually learns**.

Output Weighted Sum and Final Prediction

At this stage, we already have:

- H → hidden layer output (10 × 4)
- 4 hidden neurons → feeding into **1 output neuron**

Now the output neuron must combine these signals. Hence the **Output Layer Computation:**

$$z_{\text{out}} = H \cdot W_{\text{out}} + b_{\text{out}}$$

- **Vectorized Computation**

```
# Output layer parameters
W_out = np.array([0.5, 0.3, 0.4, 0.6])
b_out = 0.1

# Compute weighted sum (vectorized)
z_out = np.dot(H, W_out) + b_out

print("z_out:\n", z_out)
```

Instead of looping over samples and looping over neurons. We use one matrix operation. All computations done simultaneously. This is ideally how real Machine Learning systems work.

Now we convert raw values into probabilities with the sigmoid function:

```
# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Convert to probabilities
y_hat = sigmoid(z_out)

print("Predicted probabilities:\n", y_hat)
```

Each value in `y_hat` represents probability that the sample is **malware (class = 1)**

Now we measure: “How wrong are we?” with BCE Loss:

```
def binary_cross_entropy(y, y_hat):
    eps = 1e-9  # prevent log(0)
    loss = -np.mean(y * np.log(y_hat + eps) + (1 - y) * np.log(1 - y_hat + eps))
    return loss

loss = binary_cross_entropy(y, y_hat)
print("Loss:", loss)
```

This is the essentially an optimization with the objective is to minimize the loss, and the **objective function** $L = \frac{1}{n}\sum_{i=1}^{n} L^{(i)}$.

Gradient Simplification

Here’s something beautiful. When using: Sigmoid activation + Binary Cross-Entropy

The gradient simplifies to:

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

This omits complex derivatives. Faster computation. More stable learning

```
dz = y_hat - y
```

This single line replaces complex derivative chains with multiple steps

Gradients for Output Layer

Now we compute how much each weight contributed to the error

- The weight gradient formula:

$$\frac{\partial L}{\partial W_{out}} = \frac{1}{n} H^T (\hat{y} - y)$$

- The bias gradient formula:

$$\frac{\partial L}{\partial b} = \text{mean}(\hat{y} - y)$$

The code computes the output gradient:

```
n = len(y)

dW_out = np.dot(H.T, dz) / n
db_out = np.mean(dz)
```

Gradient Descent Update - Now we correct the model.

$$w = w - \eta \frac{\partial L}{\partial w}$$

The code implements **parameter update with gradient descent**

```
learning_rate = 0.01

W_out = W_out - learning_rate * dW_out
b_out = b_out - learning_rate * db_out

print("Updated W_out:", W_out)
print("Updated b_out:", b_out)
```

Wrapping Everything (Backprop Function)

Now we combine everything into one clean function → **Full Backpropagation (Output Layer)**

```
def backprop_output_layer(H, y, W_out, b_out, lr=0.01):
    # Forward pass
    z_out = np.dot(H, W_out) + b_out
    y_hat = sigmoid(z_out)

    # Error gradient
    dz = y_hat - y
```

```
    n = len(y)

    # Gradients
    dW = np.dot(H.T, dz) / n
    db = np.mean(dz)

    # Update parameters
    W_out -= lr * dW
    b_out -= lr * db

    return W_out, b_out, y_hat


# Run one step
W_out, b_out, y_hat = backprop_output_layer(H, y, W_out, b_out)

print("Updated W_out:", W_out)
print("Updated b_out:", b_out)
print("Predicted y:", y_hat)
```

Look at the full loop:

$$\text{Hidden Layer} \rightarrow \text{Output} \rightarrow \text{Prediction} \rightarrow \text{Loss} \rightarrow \text{Gradient} \rightarrow \text{Update}$$

This is no longer just computation. This is a system that corrects itself. Every iteration. It predicts. It evaluates. It adjusts. And slowly, it becomes better than it was before.

6. Backpropagation — Updating the Parameters

Now, we reach the final stage of learning. Everything we built so far:

- Forward pass → produces predictions
- Loss → measures error
- Backpropagation → computes gradients

But one question remains: ***How exactly do we update the parameters?*** This is where optimization comes in.

Optimization Using Adam

At the end of backpropagation, we have gradients for all weights and gradients for all biases. But gradients alone don't change anything. We need an **optimizer**.

From SGD → To Something Smarter

The simplest update rule is **Stochastic Gradient Descent (SGD)**

$$w = w - \eta \frac{\partial L}{\partial w}$$

Move in the direction that reduces loss. Step size controlled by learning rate η. But SGD can oscillate, converge slowly, struggle when gradients vary. So we need something more stable.

Adam Optimizer — The Upgrade

Adam combines two powerful ideas:

- **Momentum → "Remember the past"**
- **Adaptive Learning Rate → "Adjust step size"**

First Moment (Momentum): $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$

Second Moment (Variance): $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

Bias Correction - Because we start from zero:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Final Update Rule

$$w = w - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Adam works better because of smooth updates (momentum), stable updates (variance scaling), and faster convergence.

- **Initialize Adam Parameters -** Before training begins, we define:

```
# Adam hyperparameters
lr = 0.01
beta1 = 0.9
beta2 = 0.999
epsilon = 1e-8
```

- **Moment Initialization**

```
# Output layer moments
m_w_out = np.zeros_like(W_out)
v_w_out = np.zeros_like(W_out)
m_b_out = 0
v_b_out = 0

# Hidden layer moments
m_w_h = np.zeros_like(W)
v_w_h = np.zeros_like(W)
m_b_h = np.zeros_like(b)
v_b_h = np.zeros_like(b)

# Time step
t = 1
```

At the beginning: No gradients observed → Everything starts at zero

- **Adam Update Function -** Now we implement one update step.

```
def adam_update(W, b, dW, db, m_w, v_w, m_b, v_b, t,
                lr=0.01, beta1=0.9, beta2=0.999, eps=1e-8):

    # First moment (momentum)
```

```
    m_w = beta1 * m_w + (1 - beta1) * dW
    m_b = beta1 * m_b + (1 - beta1) * db

    # Second moment (variance)
    v_w = beta2 * v_w + (1 - beta2) * (dW ** 2)
    v_b = beta2 * v_b + (1 - beta2) * (db ** 2)

    # Bias correction
    m_w_hat = m_w / (1 - beta1 ** t)
    v_w_hat = v_w / (1 - beta2 ** t)
    m_b_hat = m_b / (1 - beta1 ** t)
    v_b_hat = v_b / (1 - beta2 ** t)

    # Parameter update
    W = W - lr * m_w_hat / (np.sqrt(v_w_hat) + eps)
    b = b - lr * m_b_hat / (np.sqrt(v_b_hat) + eps)

    return W, b, m_w, v_w, m_b, v_b
```

- **Training Loop (Introducing Epochs) -** Now we define **epochs**.

An **epoch** is one full pass through the entire dataset.

So: 1 epoch → model sees all 10 samples once; 100 epochs → model learns from data 100 times

```
# Initialize Parameters
np.random.seed(42)

input_size = X.shape[1]
hidden_size = 4

W = np.random.randn(hidden_size, input_size) * 0.01
b = np.zeros(hidden_size)

W_out = np.random.randn(hidden_size) * 0.01
b_out = 0.0
```

```
# Adam states
m_w = np.zeros_like(W)
v_w = np.zeros_like(W)
m_b = np.zeros_like(b)
v_b = np.zeros_like(b)

m_w_out = np.zeros_like(W_out)
v_w_out = np.zeros_like(W_out)
m_b_out = 0
v_b_out = 0
```

```
# Training
epochs = 100
t = 1
```

```
for epoch in range(epochs):
    # Forward pass
    Z = np.dot(X, W.T) + b
    H = sigmoid(Z)

    z_out = np.dot(H, W_out) + b_out
    y_hat = sigmoid(z_out)

    # Loss
    loss = binary_cross_entropy(y, y_hat)

    # Backpropagation
    dz_out = y_hat - y

    # Output layer gradients
    dW_out = np.dot(H.T, dz_out) / len(y)
    db_out = np.mean(dz_out)

    # Hidden layer gradients
    dH = dz_out[:, None] * W_out
    dZ = dH * H * (1 - H)

    dW = np.dot(dZ.T, X) / len(X)
    db = np.mean(dZ, axis=0)

    # Adam Updates
    W_out, b_out, m_w_out, v_w_out, m_b_out, v_b_out = adam_update(W_out, b_out, dW
_out, db_out,
                                                                   m_w_out, v_w_ou
t, m_b_out, v_b_out, t)

    W, b, m_w, v_w, m_b, v_b = adam_update(W, b, dW, db, m_w, v_w, m_b, v_b, t)
    t += 1

    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.6f}")
```

When we fit the optimizer. We witness the loss is gradually decreasing. That means learning is happening.

- **Final Model Evaluation**

```
# Final forward pass
z_out = np.dot(H, W_out) + b_out
y_hat = sigmoid(z_out)

print("Final predicted probabilities:\n", y_hat)
# Convert to classes
predictions = (y_hat >= 0.5).astype(int)

print("Predicted classes:\n", predictions)
print("Actual classes:\n", y)
```

```
Final predicted probabilities:
[0.11486161 0.82734376 0.16367648 0.885503   0.10909123 0.87961767
0.12481414 0.88430902 0.11187792 0.86301212]
Predicted classes:
[0 1 0 1 0 1 0 1 0 1]
Actual classes:
[0 1 0 1 0 1 0 1 0 1]
```

Look at the full system now:

$$\text{Data} \rightarrow \text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backprop} \rightarrow \text{Adam} \rightarrow \text{Update} \rightarrow \text{Repeat}$$

This is no longer just math. This is a system that improves through experience.

One last important detail: If you run this multiple times, the results may slightly change

Why?

- Random initialization
- Small dataset (only 10 samples)
- Different optimization paths

But despite that, the system consistently learns. This is the moment everything clicks. Neural networks don't memorize rules. They **discover patterns** through iteration.

You can download the full code below:

multi_layer_perceptrons.py

7. Multi-Layer Perceptron using Scikit-Learn

Multi-Layer Perceptron model can be easily implemented with Scikit-Learn for both classification and regression problem.

- Multi-Layer Perceptron for classification using the malware dataset:

Download the dataset within this link (since it is large I cannot put it directly in the post): malware_dataset.csv

```
# Multi-Layer Perceptrons
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix

df = pd.read_csv("malware_dataset.csv").set_index('hash')
target = 'classification'
features = df.columns.drop(target)

# Label Encoding for the target
df[target] = df[target].map({"benign": 0, "malware": 1})

# 80% training and 20% test
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
X_train = train_df[features]
X_test = test_df[features]
y_train = train_df[target]
y_test = test_df[target]

# Feature scaling (important for Neural Networks)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build MLP model
model = MLPClassifier(hidden_layer_sizes=(64,32), activation='logistic',
solver='adam', max_iter=300, random_state=42)

# Train model
model.fit(X_train, y_train)
# Model Evaluation
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

- Multi-Layer Perceptron for regression using the Boston housing dataset:

boston_housing.csv

```
# Multi-Layer Perceptrons
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt

df = pd.read_csv("boston_housing.csv")
# Feature and target
target = 'medv'
```

```
features = df.columns.drop(target)

train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
X_train = train_df[features]
y_train = train_df[target]
X_test = test_df[features]
y_test = test_df[target]

# Feature scaling (important for Neural Networks)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build MLP model
model = MLPRegressor(hidden_layer_sizes=(64,32), activation='relu',
solver='adam', max_iter=500, random_state=42)

# Train model
model.fit(X_train, y_train)
# Model Evaluation
y_pred = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Summary

Multi-Layer Perceptron (MLP). The name sounds complex, but now it shouldn't feel that way anymore.

A **Multi-Layer Perceptron** is simply a system that takes inputs, transforms them through layers, learns from its mistakes, and improves over time

What started as basic machine learning has now evolved into a **learning system** that can model complex patterns, not just linear relationships.

This is the turning point: From writing rules to letting the model **learn the rules itself.** And that... is the essence of Neural Networks.

References

https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/

https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning

https://d2l.ai/chapter_multilayer-perceptrons/mlp.html

https://www.youtube.com/watch?v=zs6hI0OtgNw

Next Section:

50. Reinforcement Learning